Singleton Classes and Method Lookup

As you explore singleton classes, you may notice that their names are not meant to be uttered by humans. When you print it on the screen, a singleton class looks something like this:

```
obj = Object.new
obj.singleton_class  # => #<Class:#<Object:0x007fd96909b588>>
```

The diagrams in the rest of this chapter identify singleton classes with a simple # prefix. By this convention, #obj is the singleton class of obj, #C is the singleton class of C, and so on.

Armed with the singleton_class method and your new naming convention, you can now proceed with your fearless exploration of the object model. Let's go back to the "lab rat" program and define a *Singleton Method (114)*.

```
class << obj
  def a_singleton_method
    'obj#a_singleton_method()'
  end
end
```

Now for an experiment. You know that a singleton class is a class, so it must have a superclass. Which is the superclass of the singleton class?

```
obj.singleton_class.superclass   # => D
```

The superclass of obj's singleton class is D. Try adding this newfound knowledge to the diagram of the "lab rat" object model. The result is shown in Figure 7, *Method lookup with singleton classes*, on page 123.

You can see how Singleton Methods fit into the normal process of method lookup. If an object has a singleton class, Ruby starts looking for methods in the singleton class rather than the conventional class, and that's why you can call Singleton Methods such as obj#a_singleton_method. If Ruby can't find the method in the singleton class, then it goes up the ancestors chain, ending in the superclass of the singleton class—which is the object's class. From there, everything is business as usual.

Now you understand how Singleton Methods work. But what about class methods? Yes, they're just a special case of Singleton Methods, but they deserve a closer look.